Architecture Proposal: Alternate Labels and Customer Terminology

Oct 31, 2026 · @jeffrey

Summary and decision requested

Hold alternate labels and customer terminology in a governed lexicon layer beside the core vocabularies, not inside them. The core and domain assets keep preferred labels, definitions and relations only. Everything customers and Insperity call those concepts, and every concept only one customer uses, is captured, warehoused, mapped and managed in separate scoped graphs that point at the core by ID.

The result: onboarding a customer adds no triples to the core, a label change never forces a core release, one customer's terms never reach another, and every label traces to its evidence and the reviewer who approved it.

This is an evolving baseline. The same layer is meant to support configuration, customization, personalization, provisioning and insulation at scale, so every later project reuses it instead of rebuilding it.

The baseline is the set of semantic services. The review screens, provisioning, monitoring dashboards, triage and simulation are operational extensions of the target operating model, built on the services' contracts.

Decisions requested:

1. Approve the lexicon-layer pattern: scoped lexicons for labels, customer extension vocabularies for new local concepts, and a derived resolution index.
2. Approve the five-function service model (capture, warehouse, map, manage, serve) as the baseline for every customer and future project.
3. Approve a benchmark spike before build, with pass thresholds set in advance, to prove the performance and release-stability claims.

Context and problem

The SKL is the single source of truth for vocabularies, taxonomies and ontologies, and publishes governed projections to execution points such as Visier. Customers describe the same things in their own words: their handbooks, documentation, CMS pages and

data feeds carry plan names, abbreviations, job families and policy names.

Storing all of that as alternate labels on the core concepts fails in five ways:

- **Growth.** The core grows with every customer, and so do its indexes and reasoning cost.
- **Churn.** Every label change needs a core release and steward time.
- **Leakage.** One customer's terms become visible to every other customer.
- **Ambiguity.** The same word can mean different things to different customers, and a flat label list cannot say so.
- **Lost evidence.** A bare label records neither where it came from, who approved it, nor when it applies.

Some customer terms are not synonyms at all. They name something the core has no concept for, and need a home that does not widen the core.

Design principles

Nine rules govern every choice below.

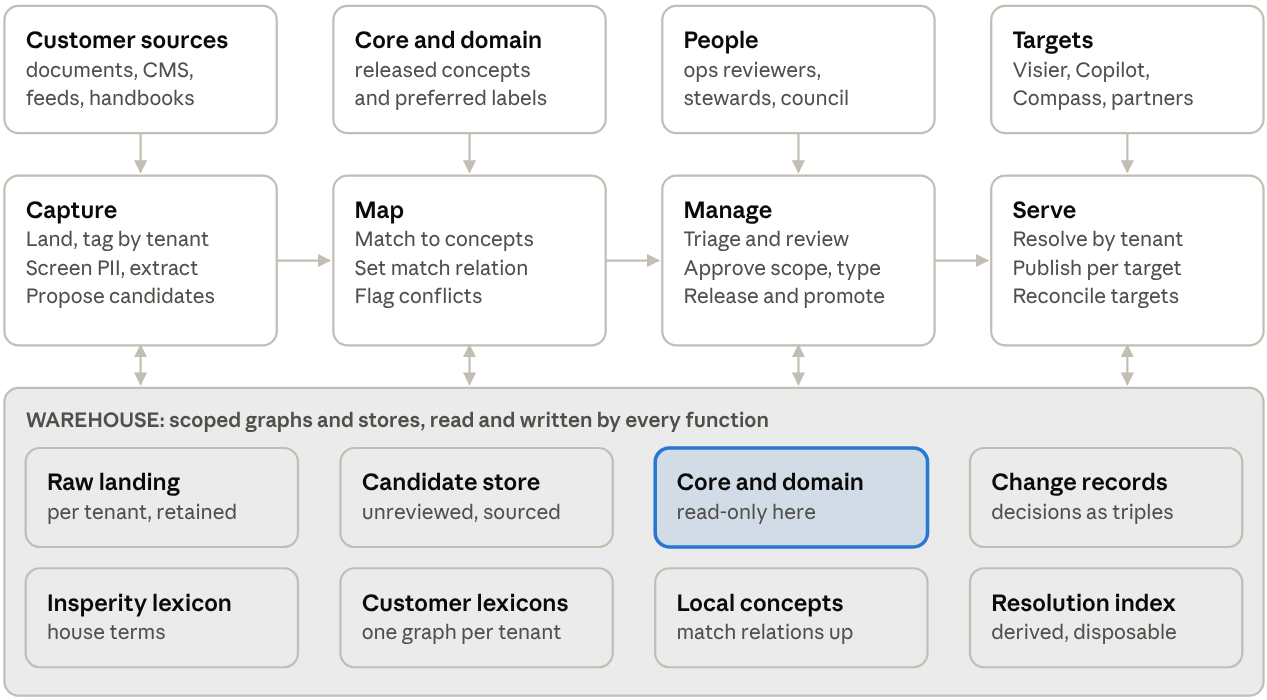
1. **The core holds meaning, not vocabulary variety.** Core and domain assets carry preferred labels, definitions and relations, plus a small vetted set of universal alternate labels.
2. **A label is a record, not a property.** Each label carries its scope, type, language, evidence, status, reviewer and dates.
3. **Scope is enforced by where a label lives.** One named graph per tenant; no query path crosses tenants.
4. **References point one way.** Lexicon and extension graphs point at core concepts by ID. Nothing is copied into the core.
5. **Machines propose, people decide.** Extraction and matching create candidates; reviewers approve them. This follows the proposal's rule to automate the mechanical and govern the judgemental.
6. **Derived is not source.** The resolution index is rebuilt from the graphs and never edited.
7. **Every change is traceable.** Source, evidence span, reviewer, SKL release and publish record are kept for every label and mapping.
8. **One contract per function, an adapter per source or target.** The design stays stack-independent and reversible.

9. **No direct access to a graph.** Repositories are never exposed to people, other systems or third parties. Every read and write goes through a semantic service or a provisioned screen that logs it and applies the governance rules.

Architecture overview

Five functions share one warehouse. Capture brings content in, Map relates it to the core, Manage approves it and Serve delivers it. The warehouse holds every store, and the core and domain graphs are only read.

Customer terms land in scoped graphs, never in the core



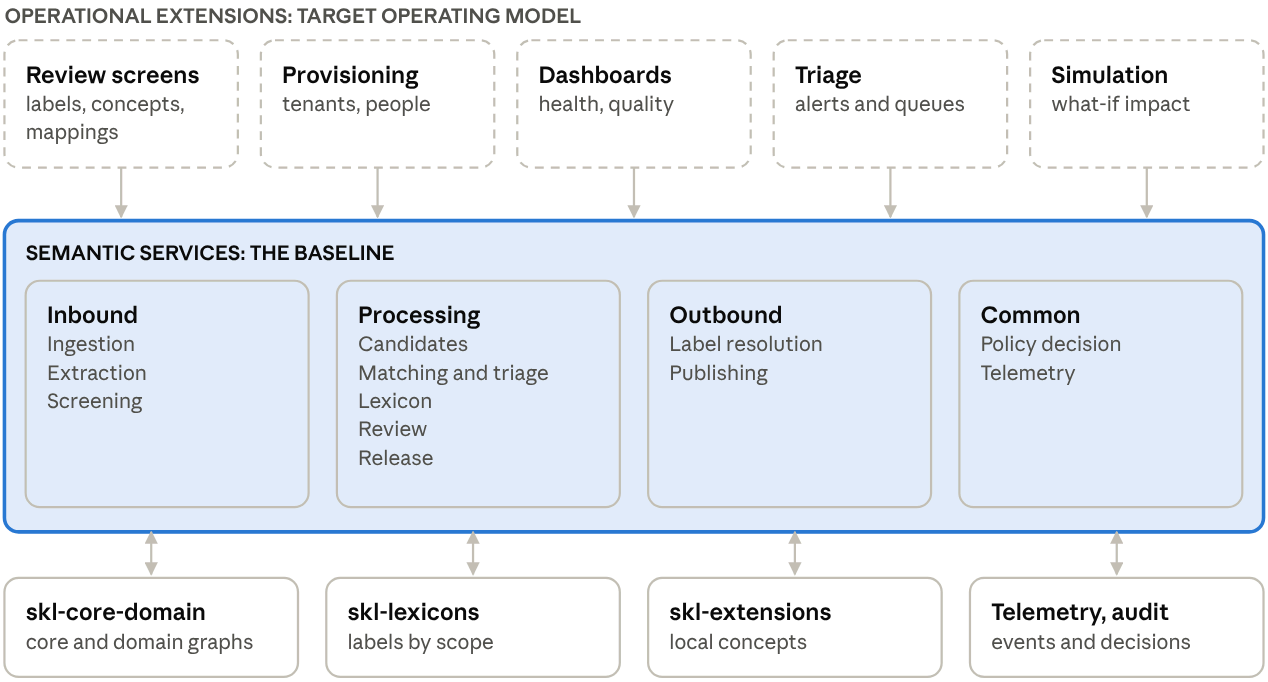
architecture · 4 functions, 1 shared warehouse

Read the top row as inputs, the middle row as the four functions in flow order, and the bottom band as the warehouse they share. Customer terms enter at Capture and leave through Serve. Nothing is written to the core and domain graphs by this flow; they change only through the council's release process.

The semantic services

The semantic services are the baseline. Every graph is reached only through them, and every later project, screen and dashboard is built on their contracts. Each service has one contract and, where it meets a source or a target, an adapter.

Extensions build on the services, and only the services touch graphs



Sources enter through Inbound and targets are fed by Outbound. Extensions call services; none touches a graph.

baseline services and operational extensions · 3 layers

Group	Service	Responsibility
Inbound	Ingestion	Lands each item with tenant tag, provenance and change capture, through one connector per source type
Inbound	Extraction	Recovers text, structure and language, with OCR for scans
Inbound	Screening	Detects personal and sensitive content before any other step, and fails closed
Processing	Candidates	Extracts terms, defined terms and drafted definitions with evidence, then normalizes and merges variants

Group	Service	Responsibility
Processing	Matching and triage	Matches candidates to concepts, proposes match relations, and classifies each as a label, a local concept, a core request or a rejection
Processing	Lexicon	Writes label records and extension vocabularies to scoped graphs, validates their shapes and detects conflicts
Processing	Review	Holds the lifecycle state machine, assignments, decisions and the audit trail
Processing	Release	Runs validation, versioned releases, promotion and rollback across environments, and records changes as triples
Outbound	Label resolution	Answers tenant-scoped lookups from the derived index
Outbound	Publishing	Projects, validates, delivers and reconciles releases to targets through adapters
Common	Policy decision	Decides each request from persona, job step, resource and context
Common	Telemetry	Receives events from every service for monitoring, triage and simulation

Every service shares the same rules:

- Each request carries a caller identity and a tenant context.
- A policy decision is made and logged before any read or write.
- Graph access is parameterized. No service accepts a raw query.
- Each service emits events for request, decision, outcome and duration.
- The contract is versioned, and changing it is a governed change.

Screens, provisioning, dashboards, triage and simulation are operational extensions of the target operating model. They call these services and never touch a graph or a repository.

Capture

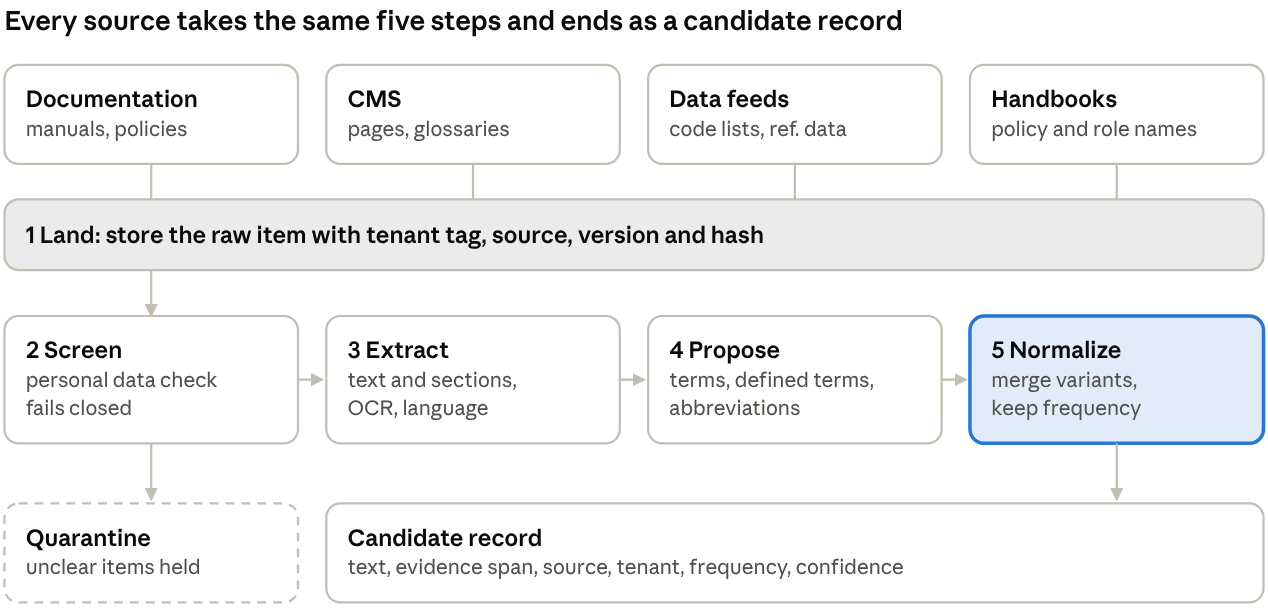
Capture brings each customer's source material in once, tags it by tenant, screens it, and turns it into candidate terms with evidence. Four source types are in scope.

Source	What it yields	Intake
Customer documentation	Defined terms, abbreviations, product and plan names	Upload or file-share intake; version detection
Content management system	Page titles, glossaries, taxonomies, navigation labels	API or export; incremental sync; page-level provenance
Customer data feeds	Code lists: job titles, departments, pay and benefit codes	File, SFTP or API; declarative R2RML-style mappings to the candidate model
Employee handbooks	Policy names, defined-terms sections, program and role names	Section-aware parsing; OCR for scans

Every source passes the same five steps:

1. **Land.** Store the raw item in the tenant's landing zone with tenant tag, source, version and hash.
2. **Screen.** Detect personal data and sensitive content before anything else runs. The screen fails closed: unclear items are quarantined.
3. **Extract.** Recover text and section structure, detect language, and run OCR with a confidence score on scans.
4. **Propose.** Extract candidate terms, defined-term and glossary patterns, and abbreviations. A definition is drafted only when a source span supports it.
5. **Normalize.** Collapse case, morphology and locale variants, and merge duplicates, keeping frequency and context.

The output is a candidate record: text, evidence span, source, tenant, frequency and confidence. Raw content stays in tenant-scoped storage. Only candidates move on.



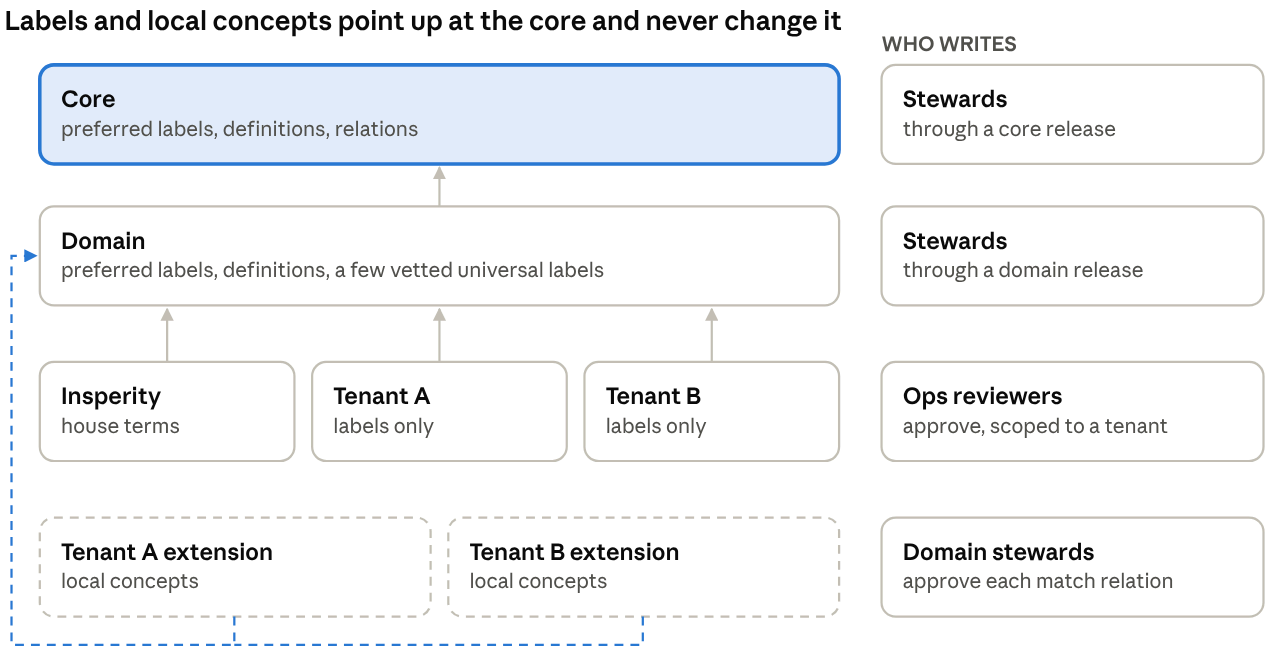
Raw content stays in the tenant landing zone. Only candidate records move on.

capture pipeline · 4 sources, 5 steps

The four sources feed one landing step; only the normalized candidate record leaves capture, and anything the screen cannot clear is held in quarantine.

Warehouse

The warehouse keeps each kind of content in its own store, so that customer terminology can grow without touching the core. Working names below follow the core and domain naming convention and are fixed in the naming task (W1-03); the SKL holds vocabularies, taxonomies and ontologies only, with no modules.



Solid arrows: a reference by ID. Dashed: a match relation (exact, close, broad or narrow). Nothing in the lexicon or extension rows is copied into the core.

scopes and references · who writes where

Each row is its own graph. References run upward by ID, and only stewards writing a release can change the core or domain rows.

Store	Holds	Written by	Release behavior
Raw landing (per tenant)	Source files, extracts, hashes	Capture	Retained by policy; never loaded into a graph
Candidate store	Unreviewed candidates with evidence	Capture, Map	Working data; never released
Core and domain graphs	Preferred labels, definitions, relations, a small vetted set of universal labels	Stewards, through the change lifecycle	Versioned core and domain releases
Insperity lexicon (lexicon-insperity)	House terms valid for all tenants	Reviewers	Own lexicon version

Store	Holds	Written by	Release behavior
Customer lexicons (lexicon-customer-tenant)	Labels valid for one tenant only	Reviewers	Own version per tenant
Customer extensions (vocabulary-customer-tenant)	New local concepts with match relations to the core	Stewards	Own version per tenant
Resolution index	Label to concept, by scope	Rebuilt from the graphs	Disposable; rebuilt on release
Change records	Every decision and release, as triples	Manage	Append-only

Repositories and access

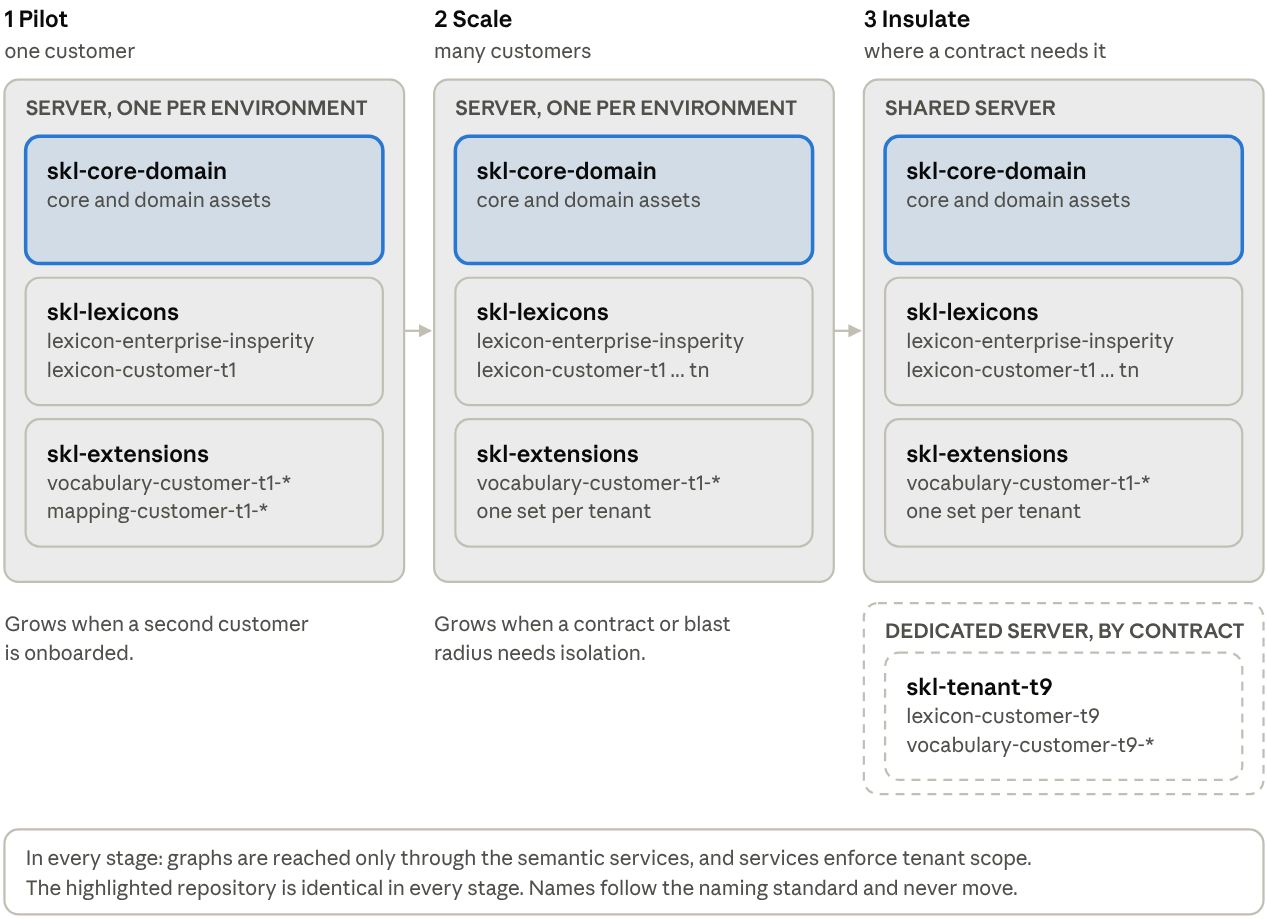
Assuming GraphDB, one server per environment (dev, test, stage, prod) hosts a few repositories. They are split by reasoning and release cadence, not by tenant.

Repository	Holds	Reasoning	Why separate
Core and domain	Core, domain and operational-domain vocabularies, taxonomies, ontologies and mapping graphs, one named graph per asset	On, as the ontologies need	Released and reasoned together
Lexicons	The Insperity lexicon and one named graph per tenant lexicon	Off or minimal	Labels stay out of reasoning and out of core releases
Customer extensions	Local concepts and their match relations, one named graph per tenant	Light	Own release and deletion per tenant

The raw landing zone, the candidate store and the resolution index need not live in GraphDB.

No repository is ever exposed to people, other systems or third parties. Each is reached only by a semantic service running under a service identity, and people work only through provisioned screens. Tenant isolation is therefore enforced by the services, with the named graph as the unit of scope. A repository per tenant, or per group of tenants, is the escalation path if a customer contract or a blast-radius concern requires it.

The topology grows by splitting repositories, and graph names never change



evolving graph topology · 3 stages

The core and domain repository is the same in every stage. Growth happens in the lexicon and extension repositories, and only where a contract or blast radius requires does a tenant move to a dedicated repository or server. Because graph names carry no tenant name, environment or repository, a move changes only the catalog record. In the diagram, t1, tn and t9 stand for tenant IDs.

The label record

A label is a resource in its own right, modeled in the style of SKOS-XL, with provenance terms from PROV-O and SHACL shapes that validate each record. This is what makes a label scopable, dateable and auditable without touching the concept it names.

Field	Holds
Label text and language	The literal form, in the language it is written
Concept	The ID of the core, domain or extension concept it names
Type	Alternate, hidden (match only, never shown), abbreviation, misspelling or deprecated
Scope and tenant	Customer, Insperity or domain; the tenant ID for customer labels
Evidence	Source, document or page, span, and how often it occurs
Confidence and triage class	The machine score and the class it proposed
Status	Proposed, in review, approved, published, rejected or deprecated
Reviewer and time	Who decided, when and why
Effective dates and release	When it applies and the SKL release that carried it

Map

Map relates every candidate to the core and domain concepts, proposes where it should live, and flags anything ambiguous. It proposes; it never approves.

Matching. Lexical matching runs first, against preferred labels and existing approved labels after normalization. Embedding-assisted similarity runs second and always returns a confidence score. Each proposal carries a suggested match relation.

Match relations. A customer concept is linked to the nearest core or domain concept with one of four relations.

Relation	Use when
Exact	The customer concept means the same as the core concept
Close	The meanings overlap but are not identical
Broad	The core concept is wider than the customer concept
Narrow	The core concept is more specific than the customer concept

A synonym, abbreviation or misspelling is not a match relation. It is a label on the concept it names. For example, an abbreviation for reduction in force becomes an abbreviation-type label on that concept.

Conflicts and homonyms. Map flags four cases and never resolves them on its own:

- The same label points to different concepts in one scope.
- A label equals another concept's preferred label.
- The same label has different meanings in two tenants. Both stay scoped; the pair is flagged as evidence for the reviewers.
- A candidate has no close match. It goes to triage as a possible local concept.

Extension vocabularies. A local concept has an ID, a definition, a status and its match relations. Concepts with several parents follow the proposal's multi-parent rule: one canonical concept, with fully qualified terms for each extra parent context. Match relations are re-checked on every core release, because a core change can leave a match stale.

Data-feed codes map through declarative R2RML-style mappings into the same candidate model, so a code list and a handbook paragraph reach review in one form.

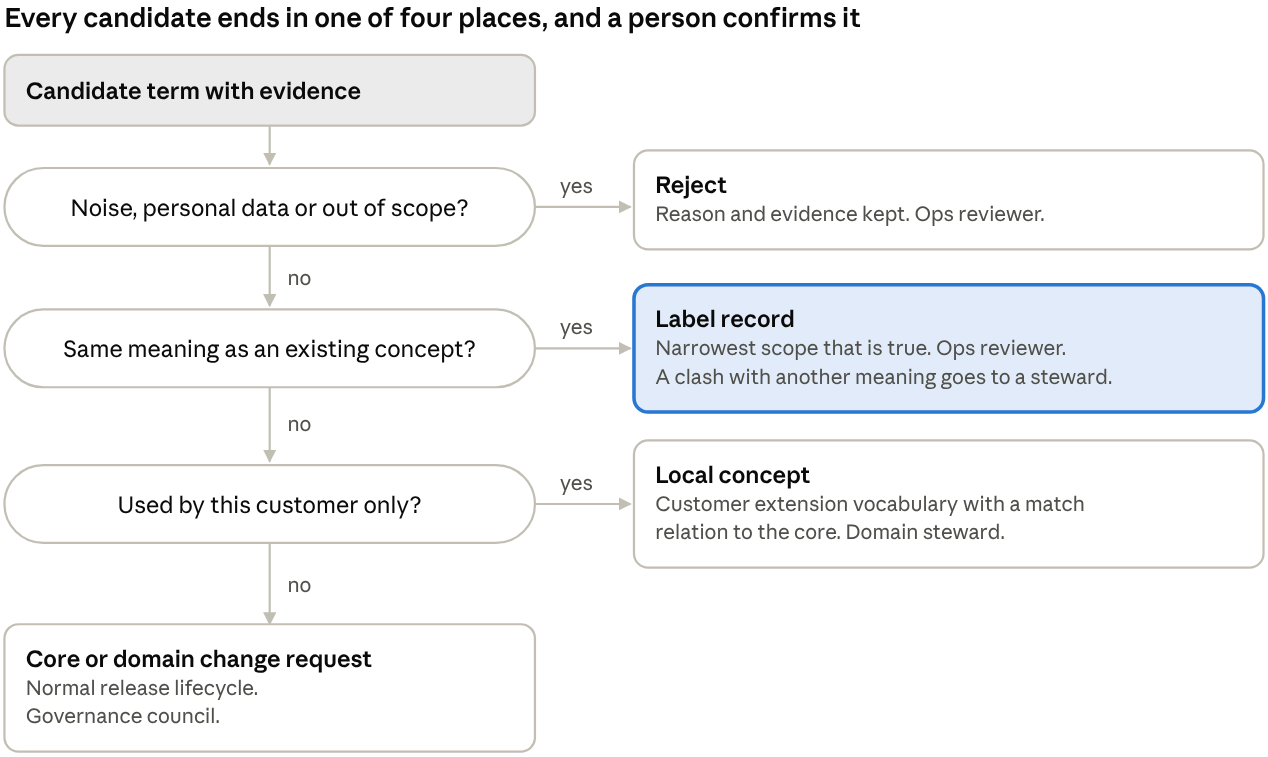
Manage

Manage turns proposals into approved, scoped content and keeps it healthy. Operational staff maintain labels, vocabularies, taxonomies, ontologies and mappings only through the provisioned screens below, within the scope their persona and job allow. The screens are an operational extension of the target operating model; the Review and Lexicon services

beneath them are the baseline. Every edit is a proposal that is validated, reviewed, approved and released, never a direct write. A change beyond a persona's scope escalates to the next persona.

Triage

Each candidate is triaged into one of five destinations, with the evidence beside it.



Rounded shapes are decisions. The service proposes each answer; a person confirms it.

triage flow · 3 decisions, 4 destinations

The table below gives the same rules with the approver for each destination.

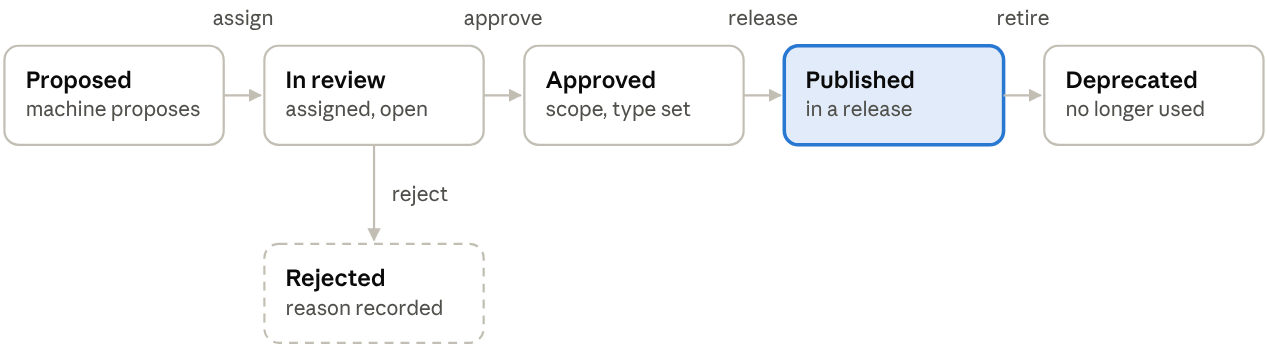
If the term is	It goes to	Approved by
The same meaning as an existing concept: a synonym, abbreviation, plural, locale variant or misspelling	A label record in the narrowest lexicon in which it is true	Ops reviewer
A homonym: the label already means something else in this scope	Stays scoped, with the conflict flagged and a disambiguation note	Domain steward

If the term is	It goes to	Approved by
A new concept used by this customer only	The customer extension vocabulary, with a match relation to the core	Domain steward
A new concept of general value, or a label seen across several customers or in regulation	A core or domain change request through the normal lifecycle	Governance council
Noise, out of scope, or personal data	Rejected with a reason; the evidence is kept	Ops reviewer

Label lifecycle

A machine-proposed label starts as **Proposed**. A reviewer moves it to **In review** and then **Approved**, or **Rejected**. An approved label becomes **Published** only through a release, and a published label can later be **Deprecated**. Every transition records who, when and why.

A label reaches users only through a release



Every transition records who, when and why.

label lifecycle · 5 states, 1 exit

Review and data-entry screens

Screen	What operations do there
Candidate queue	Filter by customer, source type, triage class, confidence and age; assign; watch the service-level timer

Screen	What operations do there
Evidence viewer	See the source snippet with the term highlighted, the document or page, and other occurrences
Concept picker	Choose among suggested matches with confidence, definition and hierarchy position
Alternate-label editor	Set text, type, language, scope, effective dates and notes; accept, edit or reject
Conflict panel	Resolve homonyms, clashes with preferred labels and cross-tenant duplicates
Local-concept creator	Create an extension concept, its definition and its match relation
Lexicon browser	Browse, search, edit, deprecate, and bulk import or export labels by scope
Escalation and promotion inbox	Send core change requests to stewards; route label promotion to the council
Reviewer dashboard	Track backlog, age of the oldest item, throughput, reviewer agreement and service levels

Promotion, demotion and releases

A label that recurs across customers, or appears in regulation, can be promoted to a domain or core vocabulary as a universal alternate label. The council approves it and it ships in a core release. A promoted label that later conflicts is demoted the same way. Lexicon changes release on their own cadence, per scope, through the same dev, test, stage and prod pipeline. A lexicon release never changes the core version. Decisions and releases are recorded as triples.

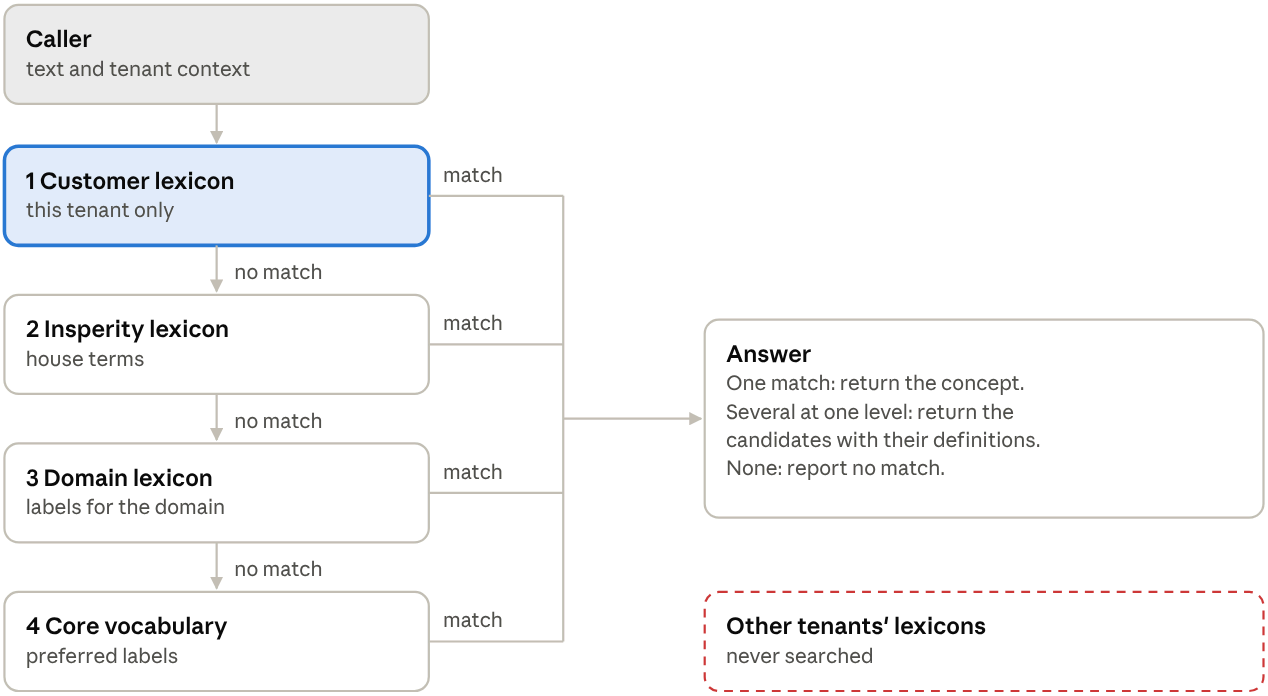
Serve

Serve makes approved terminology usable at run time and in downstream tools, without making the core carry it.

Label Resolution Service. Callers pass text and a tenant context. The service looks first in that customer's lexicon, then the Insperity lexicon, then the domain lexicon, then the core preferred labels. It never searches another tenant's graphs. Hidden labels match but are

never displayed. If more than one concept matches at the same level, the service returns the candidates with their definitions so the caller can disambiguate. It answers from a derived index that is rebuilt on each release; it does not query the ontology graphs on the hot path.

The first level with a match answers, and other tenants are never searched



Each level is read from a derived index, rebuilt on every release.

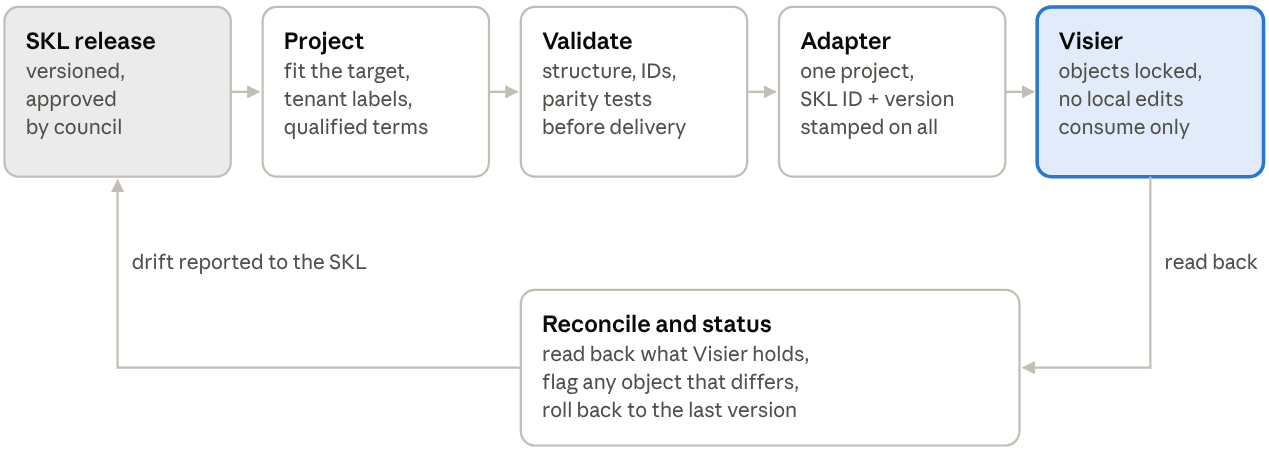
label resolution order · 4 levels

Consumers. Search, Copilot, Compass, HRScale and partner APIs resolve terms through the same service, so one approved label behaves the same everywhere.

Publishing. The outbound services follow the proposal's contract: publish, validate, reconcile, status and rollback, with one adapter per target. For Visier, the proposal publishes released vocabularies and taxonomies as dimension members and property values, each stamped with its SKL ID and version. Synonyms and definitions stay in the SKL.

Customer terminology reaches Visier through a display-label rule, proposed here for confirmation: each member shows the tenant's approved label where one exists, and the preferred label otherwise. The full set of alternate labels never leaves the SKL. Whether Visier can hold a tenant-specific name for a published member is not yet confirmed. Task W6-01 asks Visier, and if the answer is no, the preferred label is published and customer terms resolve in Copilot, Compass and search instead.

One contract publishes a release and reads back what the target holds



Proposed display rule: send the tenant label if approved, otherwise the preferred label. Alternate labels, ontology, rules and mappings stay in the SKL.

outbound publishing · 5 steps, 1 read-back loop

The flow follows the publishing contract in the SKL proposal. Visier consumes and never edits; any difference found on read-back is drift and is corrected in the SKL.

Tenant isolation, security and privacy

Customer documents and terms are confidential to the customer. The design protects them in six ways.

- **No direct access.** No repository is exposed to people, other systems or third parties. Graphs are reached only through the semantic services, and people work only through provisioned screens that log and audit every action and apply the governance rules. The store's own workbench and admin accounts are closed to normal use; any break-glass access is time-bound and logged.
- **Tenant tag from the first byte.** Every item carries a tenant ID from landing onward, and every query carries a tenant context.
- **Isolation enforced by the services.** Each tenant's lexicon and extension vocabulary is its own named graph, and the services build the graph restriction into every query from the caller's tenant context. Services accept no raw queries from callers or screens, and read and write use separate service accounts. Graph-level controls in the store, where available, are a second layer, not the first.

- **No cross-tenant path.** There is no query that spans tenants. The one exception is promotion: a label recurring across customers is counted by the review service, which shows reviewers only the count, never another tenant's evidence.
- **Fail-closed screening.** Personal and sensitive content is detected, redacted or quarantined before extraction. A red-team set tests it before the pilot gate.
- **Audit and exit.** Every read and write is logged. Offboarding a customer removes its landing zone, graphs and index entries without any change to the core, which is a direct benefit of keeping its terms out of it.

Written permission to use each customer's documents and feeds, a retention rule and the data-use terms are conditions of onboarding, and are settled with security and legal in the first weeks of the plan.

Provisioning and operational personas

This section describes an operational extension of the target operating model, built on the semantic services. Access to every function is granted by persona and job, not by a list of who may touch which graph. The Personas & Jobs model will supply the fine-grained identity and access controls, which go beyond traditional ACLs.

Operational personas

Persona	Jobs it performs	Works on
Tenant provisioner	Create, suspend and remove a tenant; rotate service credentials	Tenant records and graphs, never content
Onboarding lead	Register a customer's sources, confirm data permission, run onboarding	One tenant's sources and ingest reports
Ops reviewer	Decide candidates, edit labels, raise escalations	Candidates and label records for assigned tenants
Domain steward	Approve match relations and local concepts; maintain vocabularies, taxonomies and mappings	Assets in the domains they steward
Ontologist	Propose changes to ontologies, triage rules and naming	Ontology assets, through the lifecycle

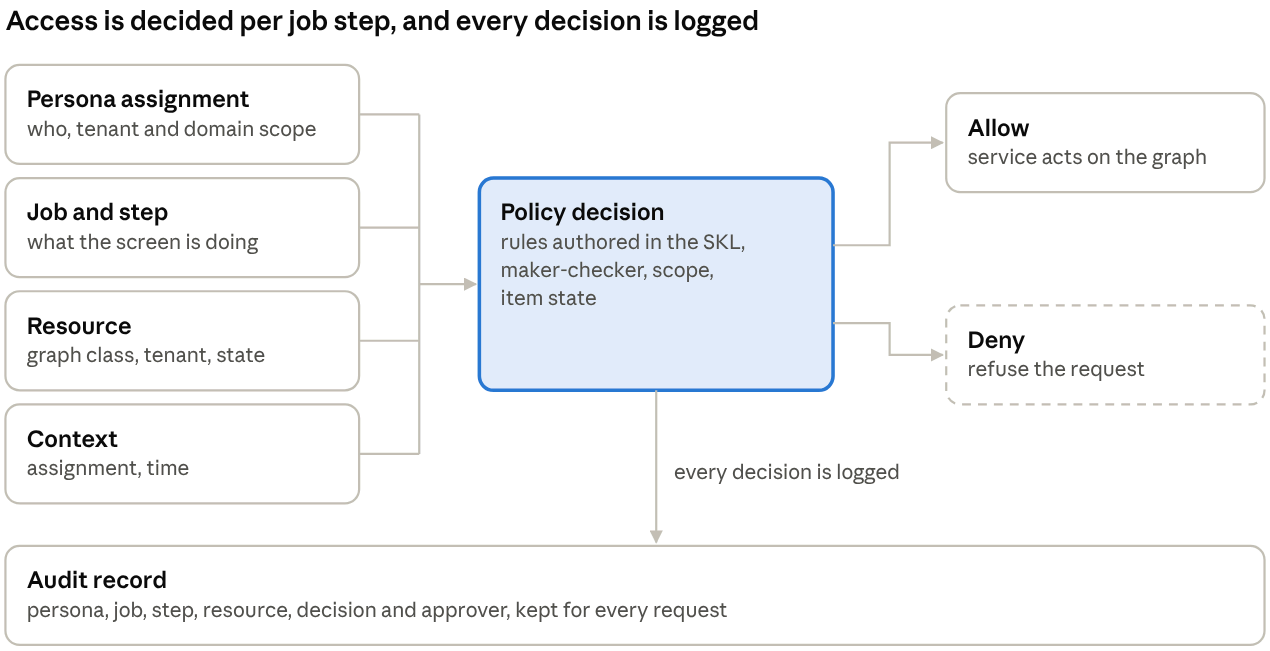
Persona	Jobs it performs	Works on
Platform operator	Run pipelines, releases, rollback and drift response	Services and releases, never content
Council member	Approve releases, label promotion and cross-domain conflicts	Release and promotion decisions
Auditor	Read the audit trail and decision records	Records only

The list is a proposal. The persona and job catalog is owned by Personas & Jobs, and each persona's jobs are broken into job steps that the screens and services present.

How access is decided

Each request carries four things: the person's persona assignments; the job and job step the screen or service call is performing; the resource, meaning graph class, tenant, domain and item state; and the context, such as assignment and time. The service makes the decision from rules authored in the SKL like other governed rules, and logs it with the persona, job and step that authorized it. These examples cannot be expressed as a list of permissions:

- A reviewer can decide only items assigned to them, in tenants they are provisioned for, while the item is In review.
- Nobody approves their own change: the persona that proposes cannot be the one that approves.
- A steward's approval power ends at the domains they steward.
- A person holding two personas has each persona's rights only while doing that persona's jobs.
- Elevated access is time-bound, logged and reviewed.



access decision · 4 inputs, 1 policy decision, 1 audit record

Provisioning

Provisioning is itself a persona job. It runs through the same provisioned screens and under the same audit and governance rules.

1. **Provision a tenant.** Register the tenant; create its lexicon and extension graphs, indexes and review queue; bind its policies and persona assignments; confirm written data permission; activate. Deprovisioning reverses each step and removes the graphs, raw store and index entries.
2. **Provision a person.** Assign one or more personas with a tenant and domain scope and an end date; complete the persona's training and calibration; activate. Assignments are reassessed periodically, and a mover's or leaver's assignments are revoked at once.

Performance and scale

The core stays fast because customer growth happens elsewhere. Three design choices carry that claim:

- Labels are annotations, not axioms, and live in graphs outside the ontology graphs, so reasoning never loads them.
- Run-time lookup reads a derived index, not the graphs, so lookup cost does not depend on ontology size.

- Each tenant is a separate graph, so loading, reindexing and deleting one customer touches no other.

These are claims until measured. Before the council approves the design, a benchmark spike (task W2-02) builds a synthetic core and holds the same labels two ways: in the core, and in scoped lexicons. It measures load time, reasoning time, query time, index size and release size at rising label volumes and tenant counts. Pass thresholds are agreed first (task W1-04) so the result cannot move them. The scale test in task W7-03 repeats the measurement on the built system.

Measure	Target	Tested in
Customer-driven triples added to the core per onboarding	0	Lexicon builder and second-customer onboarding
Core load and reasoning time at the top test volume	Within a tolerance set before the spike	Spike W2-02, scale test W7-03
P95 label lookup time	Threshold set from the spike	Spike W2-02, Label Resolution Service
Cross-tenant label results	0	Isolation tests
Core releases triggered by a label change	None	Release pipeline
Label records with source, reviewer and release	100%	Review service

Customer counts and labels per customer are not yet known, so the spike uses synthetic parameters, which the decision record states. The design is revisited at the final gate with real volumes.

Options considered

Options B and C together, chosen by the triage rule, are recommended. Option A is kept only for a small vetted set of promoted labels.

Option	How it works	Core size and speed	Tenant confidentiality	Handles new customer meaning	Verdict
A. Labels on core concepts	Every synonym is an alternate label on the core concept	Weak: grows with every customer	Weak: all tenants see all labels	Weak	Reject for customer labels
B. Scoped lexicons	Labels are records in one graph per scope, pointing at core concepts	Strong: core unchanged	Strong: scope is the graph	Weak: names existing concepts only	Adopt for label-only terms
C. Customer extension vocabularies	New local concepts with match relations to the core	Strong: core unchanged	Strong	Strong	Adopt for new local concepts
D. A copy of the core per customer	Fork the core and edit labels in the fork	Weak: copies multiply	Strong	Partial	Reject: copies drift and every core fix must be merged into each

B needs a resolution service and a derived index at run time. C needs steward review of each match. Both costs are accepted because they are paid once per platform or once per concept, while A and D charge every customer again and again.

Operating model

The roles come from the SKL proposal. Three are added here because the lexicon layer needs them.

Role	Accountable for
Semantic architect	Label architecture, triage rules, naming and ID conventions, the multi-parent rule
Domain stewards	Meaning of definitions, match relations, and core change requests
Semantic engineers	The lexicon builder, resolution service, review service and publishing adapters

Role	Accountable for
Visier model developer	Applying published releases in Visier and blocking local edits
Governance council	Approving releases, label promotion, cross-domain conflicts, and customer terms that map to nothing
Ops review lead (added)	Reviewer capacity, service levels, calibration and training
Ops reviewers (added)	Deciding candidates within their scope in the review screens
Data and NLP engineers (added)	Source connectors, extraction, matching and the gold set

The operational personas for provisioning and access, and the jobs each may perform, are set out in the section on provisioning and operational personas.

Measures, reported per customer and per release:

- Lead time from first source received to approved customer lexicon
- Customer-driven triples added to the core per onboarding (target 0)
- Candidates decided per reviewer-hour, and age of the oldest open item
- Precision and recall of extraction, matching and triage on the gold set, and agreement between reviewers
- P95 label lookup time and cross-tenant results (target 0)
- Share of labels with full provenance (target 100%)
- Drift incidents per release and parity-test failures caught before publish, from the SKL proposal

Service levels for review are set from the pilot, not guessed in advance.

Target operating model extensions

The semantic services are the baseline. The screens, provisioning and the three extensions below are operational extensions of the target operating model, built on the services' contracts. They add no route to a graph, and each can be added or changed without changing the baseline.

Extension	What it adds	Calls	Used by
Review and maintenance screens	Data entry for labels, local concepts, vocabularies, taxonomies, ontologies and mappings, as governed proposals	Review, Lexicon, Release, Policy decision	Ops reviewers, stewards, ontologists
Provisioning	Tenant and person lifecycles, persona assignments and recertification	Policy decision, Lexicon, Telemetry	Tenant provisioner, onboarding lead
Monitoring dashboards	Live and historical views of health, quality, review work, access, releases and capacity	Telemetry	Platform operators, ops review lead, auditors, council
Triage	One queue of alerts and exceptions, each with a severity, an owner and a runbook	Telemetry, Review	Platform operators, ops review lead
Simulation	What-if runs that show the impact of a change before it is made	Release, Matching, Publishing, Telemetry	Stewards, ontologists, council, program manager

Monitoring dashboards

Dashboards read telemetry, never the graphs. The measures listed above feed them.

- **Pipeline:** volume by source type and tenant, failures, quarantined items, and lead time from source to candidate.
- **Processing quality:** precision and recall on the gold set, candidate backlog, and the mix of triage classes.
- **Review work:** queue size and age, throughput, reviewer agreement and service-level status.
- **Service health:** latency and error rate for each service, including label lookup P95.

- **Access and security:** denials, break-glass use, isolation test results, and persona assignments nearing expiry.
- **Releases and publishing:** releases per environment, parity results, drift per release and rollback events.
- **Capacity:** graph size, load and reasoning time by repository, and volumes per tenant, with trends.

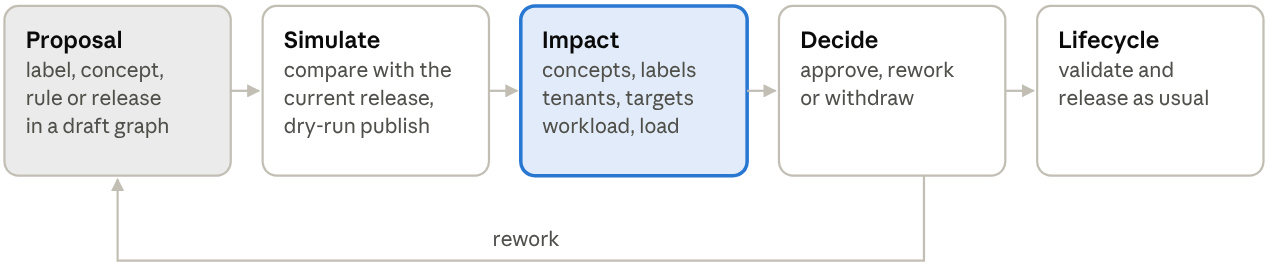
Triage

Operational triage handles what goes wrong or needs attention. Alerts and exceptions from the dashboards enter one queue: failed ingests, quarantined items, drift found by reconcile, parity failures, repeated access denials, expiring assignments and service-level breaches. Each item has a severity, an owning persona and a runbook, and is assigned, worked and closed with an audit record. This is separate from the candidate triage in Manage, which decides where a term goes.

Simulation for planning and impact

Simulation shows what a change would do before it is made. A proposed change is loaded into a draft graph, and the simulation compares it with the current release without publishing anything. Simulation workspaces are graphs named with the reserved `sim` segment in the naming standard.

A simulation shows the impact of a change before anything is published



Simulations read draft and release graphs and write only to simulation records.
No released graph is touched and nothing is published.

change-impact simulation · 5 steps, 1 rework loop

The changes worth simulating include:

- a core or domain release;
- promoting or demoting a label;

- a changed match relation;
- a new or changed access rule;
- a new tenant, or growth in volume;
- a changed projection to Visier, as a dry run.

The impact view shows the concepts, labels, mappings and tenants affected; the objects that would change in each target; new conflicts or homonyms; the reviewer workload created; and, for capacity runs, the predicted load, reasoning and lookup time. It is drawn as an impact graph with the change at the center and counts by tenant and target, so a council member can see the reach of a change before approving it.

Risks and mitigations

Risk	Mitigation
The lexicon layer adds a run-time lookup the core did not need	Derived index; P95 lookup threshold set before the spike; spike and scale test
Extraction floods reviewers with low-value candidates	Confidence floors, de-duplication, a gold set, a triage precision target, bulk actions
Reviewers approve a label at the wrong scope	Narrowest scope as the default; conflict panel; steward escalation; promotion only through the council
Extension concepts drift from the core when it changes	Re-check match relations on every core release; drift report
Customer documents hold personal or confidential content	Written permission; fail-closed screening; retention rule; red-team test
Visier cannot hold tenant-specific names, or its Dimensions API (Alpha) is not ready	Ask Visier early (W6-01); publish the preferred label; file-based or Direct Data Intake loading as fallback; labels stay in the SKL
Volumes are unknown, so the spike is not representative	Record the synthetic parameters; repeat at scale test and at the final gate with real counts
Labels go stale over time	Effective dates, deprecation, and periodic review of unused labels

Risk	Mitigation
A flaw in service-layer authorization exposes one tenant's terms to another, because the store is not the second line of defense	Parameterized queries only, separate read and write service accounts, isolation tests in the pipeline, graph-level store controls as a backstop where available
The store's own admin or workbench access bypasses the services	Private network, workbench closed to normal use, time-bound logged break-glass access, encrypted backups
Persona and job rules are too coarse or too tangled to maintain	Author them in the SKL as governed rules with test cases, review them with Personas & Jobs, and pilot them on the reviewer and steward personas first

Pilot, phasing and next steps

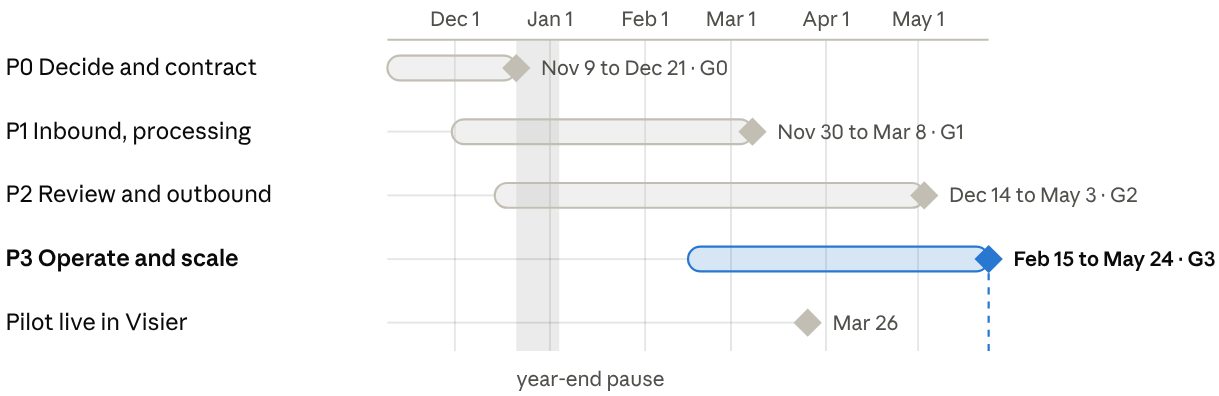
The pilot uses one customer, one instance of each of the four source types, the downturn workforce advisory vocabularies from the SKL proposal, and Visier as the first target. The dates below are planning dates from the PLM Lifecycle Dashboard · SKL Semantic Services Baseline, which starts on 9 November 2026 and assumes a pause from 21 December to 3 January.

Gate	Planning date	What is true when it passes
G0	21 Dec 2026	The label architecture is decided on benchmark evidence; the council is seated; naming, ID and multi-parent rules agreed; the inbound, processing and publishing contracts are published; pilot customer, stewards and data permission confirmed
G1	8 Mar 2027	All four source types ingested with provenance on every record; no personal-data leaks in the red team; candidates triaged into scoped lexicons; zero customer-driven triples in the core; zero cross-tenant results

Gate	Planning date	What is true when it passes
G2	3 May 2027	Pilot vocabularies live in Visier in one version with parity and reconcile checks clean; the pilot customer's lexicon reviewed by operations and published for its tenant
G3	24 May 2027	Scale test passed; a second customer onboarded from the playbook with no change to the core; a second target live; runbooks drilled

Four overlapping phases take the baseline to operation by late May 2027

Planning dates from the project plan; diamonds are gates



delivery timeline · 4 phases, 4 gates

Phases overlap because later work starts as soon as the contracts it needs exist. The grey band is the year-end pause; the dashed line marks the final gate.

Next steps:

1. Approve the three decisions in the summary.
2. Confirm the start date and staffing for the roles above.
3. Name the pilot customer and the domain stewards, and obtain written data permission.
4. Send Visier the questions in task W6-01, including tenant-specific display names.
5. Set the spike's pass thresholds before it runs.
6. Agree the operational persona and job catalog with Personas & Jobs, and the first access rules for reviewers and stewards.

Open questions and decisions requested

- Which customer is the pilot, and which CMS and feed formats does it use?
- Can Visier hold a tenant-specific name for a published member, or one name per member?
- Which graph store, review-application host and CI platform will the baseline use? The design is stack-independent, so this is a choice, not a constraint.
- How many customers and how many labels per customer should the spike and scale test assume?
- Who staffs operations review, and how many hours a week can they give it?
- Which languages beyond English appear in the pilot sources?
- What are the pass thresholds for core load, reasoning time and lookup time?
- Who approves a customer term that maps to nothing in the core? The SKL proposal leaves this open.
- Which identity provider and policy engine will make access decisions, and where will persona and job assignments be held?
- Who owns the persona and job catalog, and how are changes to it released?
- What is the break-glass procedure for the store's own admin access, and who approves it?
- Which telemetry platform will hold service events and audit records, and how long are they retained?
- Which simulation should come first: label promotion, a core release, a Visier projection or capacity planning?

Sources: the SKL proposal "Proposal: SKL as Single Source of Truth, Publishing into Visier" (29 October 2026), for the principles, roles, publishing contract, multi-parent rule, pilot scope and Visier limits, which it notes come from public documentation and should be checked against the tenant; and the PLM Lifecycle Dashboard · SKL Semantic Services Baseline for the plan and dates. Standards named (SKOS-XL, PROV-O, SHACL, R2RML) are W3C specifications.